SPI Master-Slave & RAM

BY: Eyad Noah

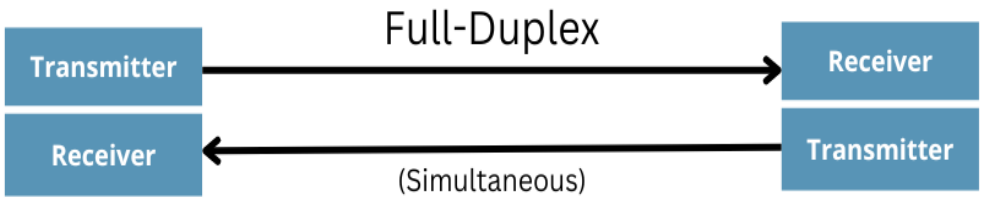
Table of Contents

Abstract

- This project details the design, implementation, and verification of a configurable Serial Peripheral Interface (SPI) communication system developed in Verilog HDL.
- The architecture features a parameterized SPI Master, an SPI Slave, and an SPI RAM module, all brought together via an integration wrapper.
- The SPI Master offers high flexibility through configurable data widths, selectable SPI modes, adjustable clock division, and a choice between MSB-first or LSB-first transmission orders.
- Meanwhile, the SPI Slave processes incoming serial commands from the master, decodes the intended operations, and interacts with the RAM to execute memory read and write cycles.
- Rigorous functional verification was successfully performed using comprehensive self-checking test benches.

Introduction

Widely adopted across embedded systems and digital integrated circuits, the Serial Peripheral Interface (SPI) serves as a prominent synchronous serial communication protocol. It enables high-speed, full-duplex data transfer between a single master device and multiple slave devices while utilizing a minimal set of signal lines.



- Unlike asynchronous protocols like UART, SPI employs a dedicated clock signal generated by the master device, which enables faster communication speeds and predictable, deterministic timing.
- This project features a complete SPI communication system designed entirely in Verilog HDL and tailored for FPGA implementation.

Project Objectives

- Design a configurable SPI Master.
- Design an SPI Slave capable of decoding SPI commands.
- Interface the slave with single port RAM.
- Support configurable SPI parameters.
- Verify correct data transmission using simulation.
- Produce synthesizable RTL suitable for FPGA implementation.

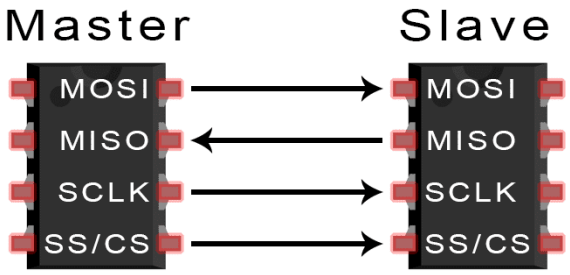
SPI Protocol and architecture

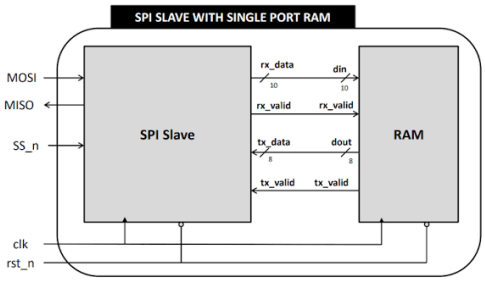
SPI communication uses four primary signals:

Signal	Direction	Description
SCLK	Master → Slave	Serial clock generated by the master
MOSI	Master → Slave	Master Out Slave In
MISO	Slave → Master	Master In Slave Out
CS_n/SS_n	Master → Slave	Active-low Slave Select

- Communication begins when the master asserts the Chip Select (CS) line low. The master then generates clock pulses while transmitting data through MOSI and simultaneously receiving data from the slave through MISO.

The implemented SPI system consists of three major modules:





Methodology

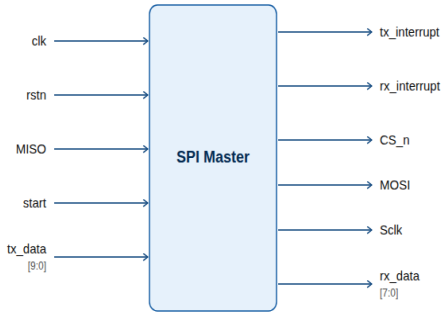
1.SPI Master

The SPI Master module acts as the core communication controller bridging the external CPU to the SPI bus:

- **Data Transmission:** It receives parallel data from the CPU and shifts it out serially to the Slave via the MOSI line.
- **Data Reception:** It captures incoming serial data from the Slave via the MISO line and assembles it into parallel words.
- **Clock and Control Management:** It generates the SPI clock (sclk) and manages chip select (cs_n) signals under SPI Mode 0.

1.Ports

Inputs	Outputs
tx_data	rx_data
start	CS_n
clk	sclk
rstn	tx_interrupt
MISO	rx_interrupt
_	MOSI



2.Modes

The Serial Peripheral Interface (SPI) protocol does not enforce a rigid clock scheme. Instead, it uses two configuration bits—**CPOL (Clock Polarity)** and **CPHA (Clock Phase)**—to define the exact timing relationship between the clock (sclk) and the data lines (mosi and miso).

CPOL	CPHA	Mode
0	0	0
0	1	1
1	0	2
1	1	3

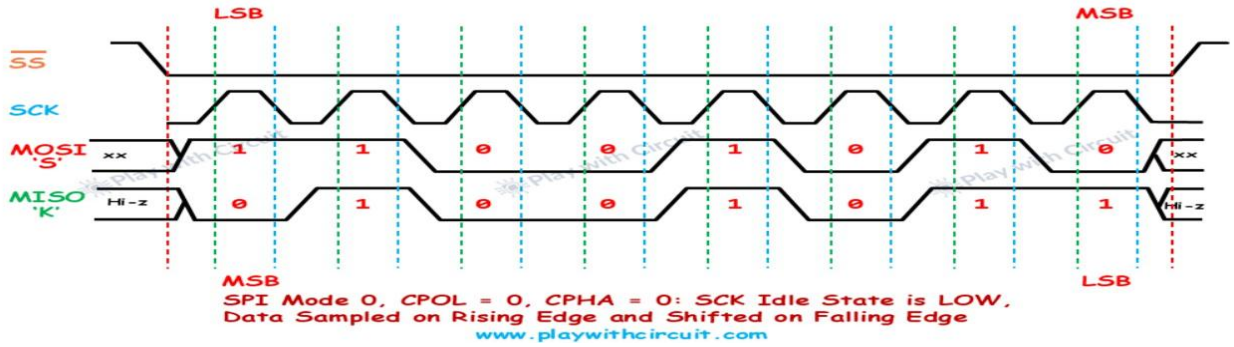
- **SPI Mode 0 (CPOL = 0,CPHA = 0):** Clock idles low; data shifts on the falling edge and is sampled on the rising edge. This is the most widely used mode in industry and the exact mode utilized by our design.

- **SPI Mode 1 (CPOL = 0,CPHA = 1):** Clock idles low; data shifts on the rising edge and is sampled on the falling edge.

- **SPI Mode 2 (CPOL = 1,CPHA = 0):** Clock idles high; data is sampled on the falling edge and shifts on the rising edge.

- **SPI Mode 3 (CPOL = 1,CPHA = 1):** Clock idles high; data shifts on the falling edge and is sampled on the rising edge.

- **Most Widely Used Mode:** SPI Mode 0 (CPOL = 0,CPHA = 0) is by far the most commonly adopted configuration across industry microcontrollers, sensors, and peripheral devices
- **Application in Our Design:** Both the SPI Master and Slave modules developed in this project are configured and verified to operate strictly using **SPI Mode 0**.

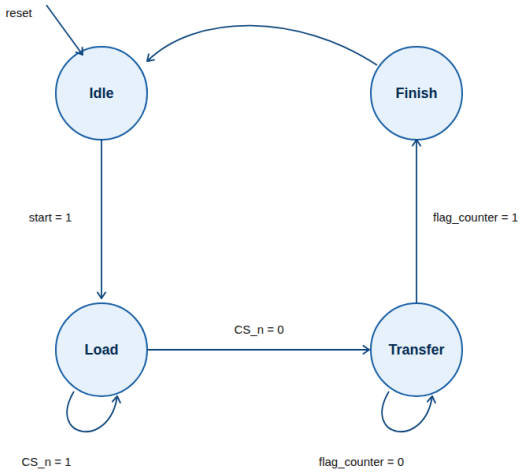


3.FSM

The SPI Master operates using a four-state finite state machine.

IDLE

- Waits for the Start signal.
- CS_n remains high.
- sclk remains at idle polarity.



LOAD

- Master send first MOSI
- Loads shift data and transmit it into the shift register.
- Clears receive register.
- Initializes counters.
- Activates Chip Select.

TRANSFER

During this state:

- Master generates the SPI clock.
- SCLK toggles.
- Master send data to Slave.
- Master sampled data from Slave.
- At the end of this state all counters are returned to 0

FINISH

- Returns to IDLE.

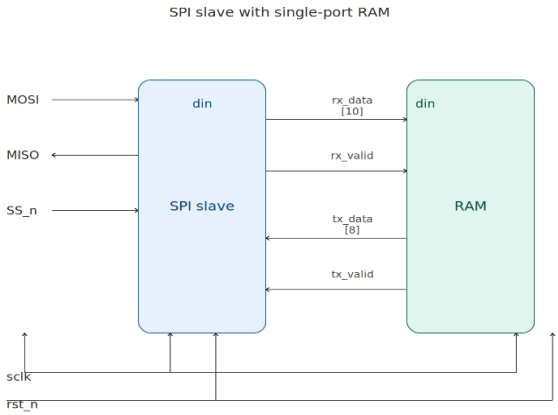
2.SPI Slave & RAM

The SPI Slave module acts as the core communication bridge between the SPI Master and the internal RAM:

- **Data Reception:** It samples serial data from the Master via the MOSI line, assembles it into a parallel word, and routes it to the RAM.
- **Data Transmission:** It takes parallel data from the RAM and shifts it out serially back to the Master via the MISO line.
- **FSM State Control:** It uses an internal finite state machine to manage write, read address, and read data operations.

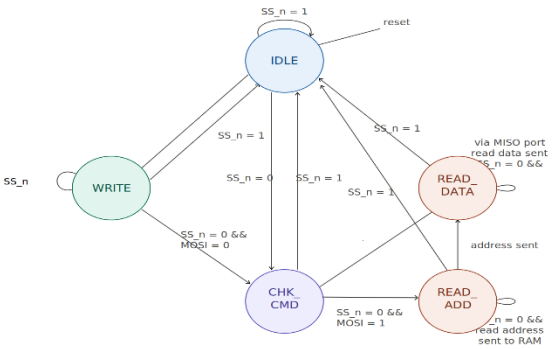
1.ports

Inputs	Outputs
sclk	MISO
rstn	_
SS_n	_
MOSI	_



2.FSM

State	Function
IDLE	Wait for CS assertion
CHECK	Decode transaction type
WRITE	Receive write command
READ_ADDR	Receive read address
READ_DATA	Transmit requested data

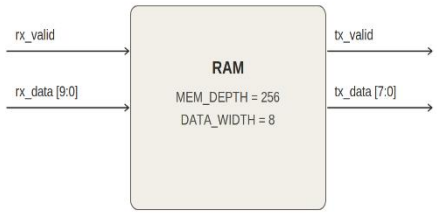


RAM

- The RAM stores 8-bit data across **256** memory locations

1.Ports

Inputs	Outputs
sclk	tx_data
rstn	tx_valid
rx_data	_
rx_valid	_



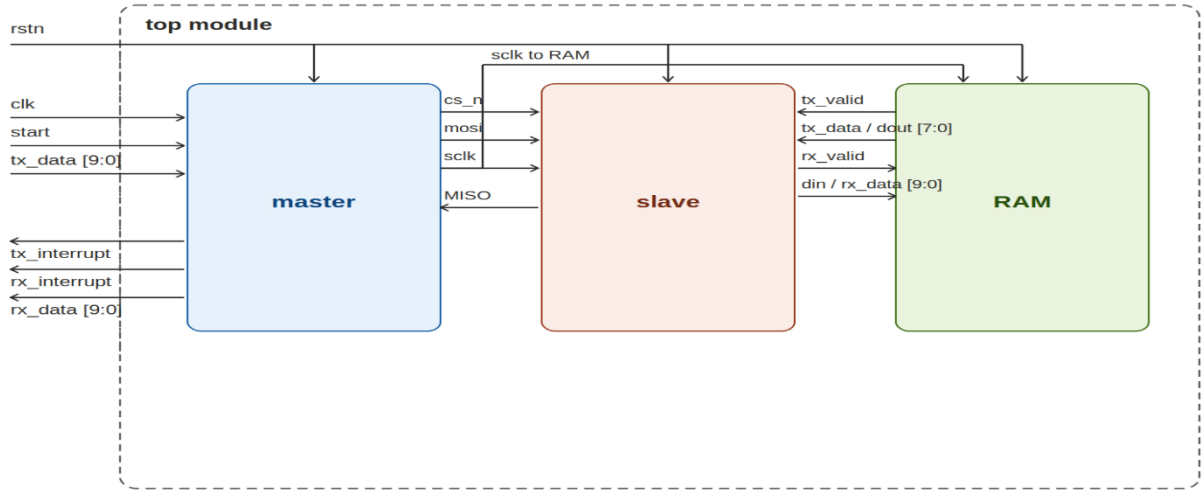
2.OP Code

Command	Function
00	Write Address
01	Write Data
10	Read Address
11	Read Data

```
    else begin
        tx_valid_unstretched <= 0;
        if (rx_valid) begin
            case (din [data_width+1:data_width])
            2'b00: write_adr <= din [data_width-1:0];
            2'b01: begin
                mem_array [write_adr] <= din [data_width-1:0];
            end
            2'b10: read_adr <= din [data_width-1:0];
            2'b11: begin
                dout <= mem_array [read_adr];
                tx_valid_unstretched <= 1;
            end
            endcase
        end
    end
end
```

Top Module

1.Block Diagram

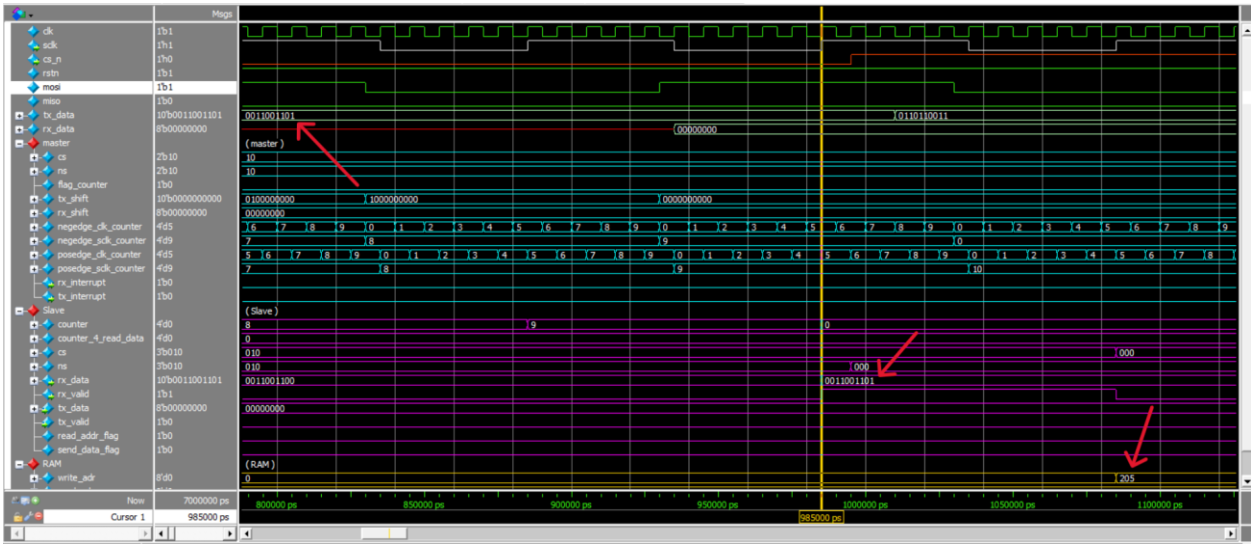


2.Ports

Inputs	outputs
clk	rx_data
rstn	rx_interrupt
start	tx_interrupt
tx_data	_

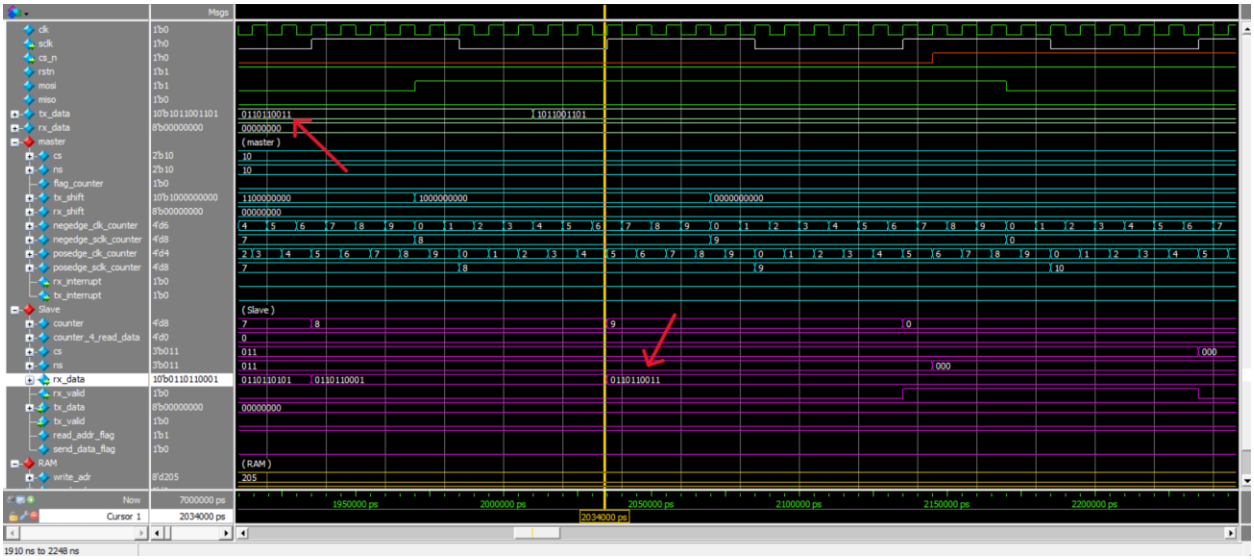
Functional Verification
1.write Address Case

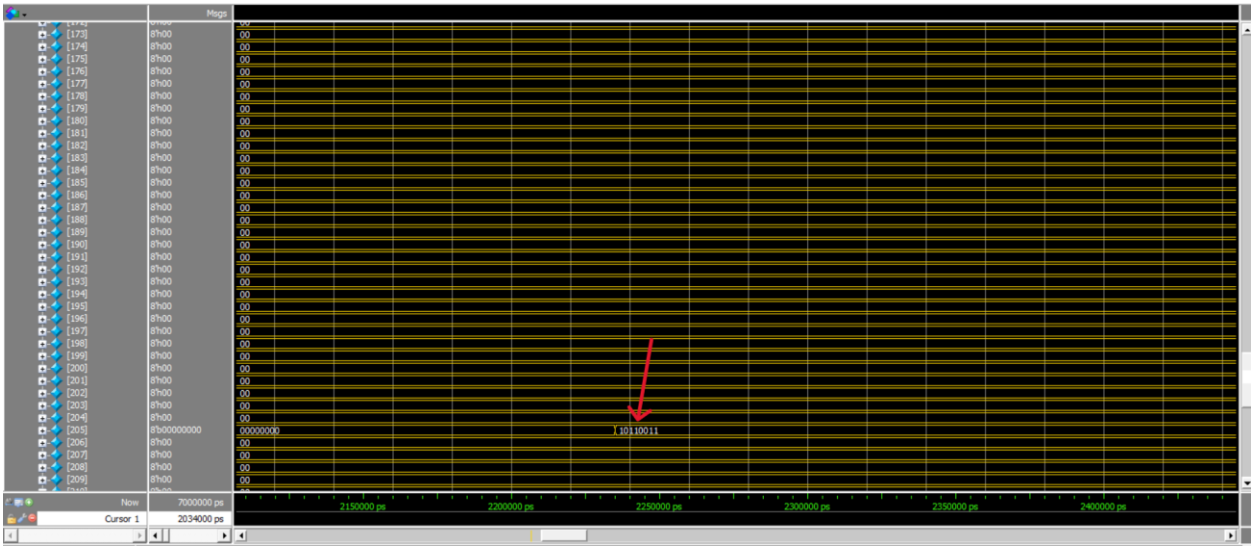
- The Master sends the specific memory location where it wants to save data. The RAM simply takes the 8-bit payload and stores it in an internal write_adr pointer register. No data is actually written to memory yet



2.Write Data Case

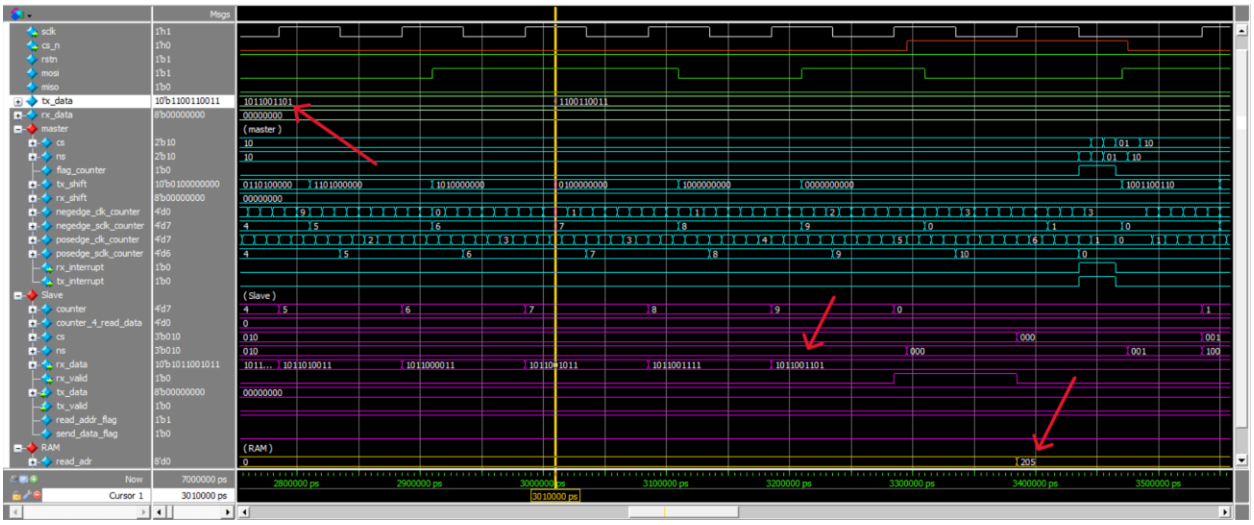
- The Master sends the actual 8-bit data payload. The RAM takes this payload and writes it directly into the memory array (mem_array) at the exact location previously saved in the write_adr register.





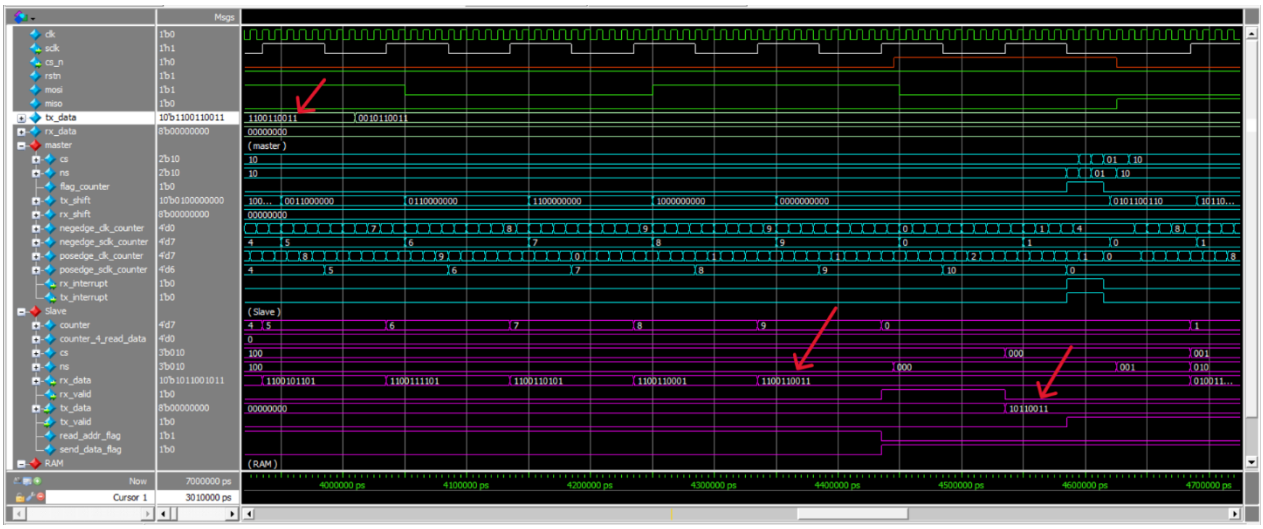
3.Read Address Case

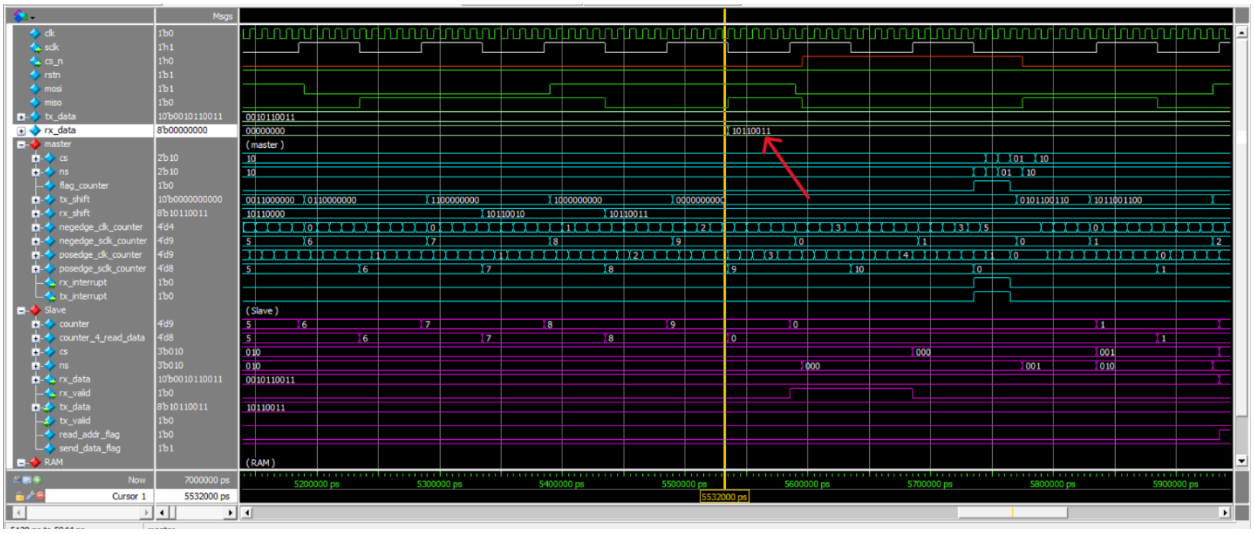
- The Master sends the specific memory location it wants to retrieve data from. The RAM takes this 8-bit payload and stores it in an internal read_adr pointer register.



4.Read Data Case

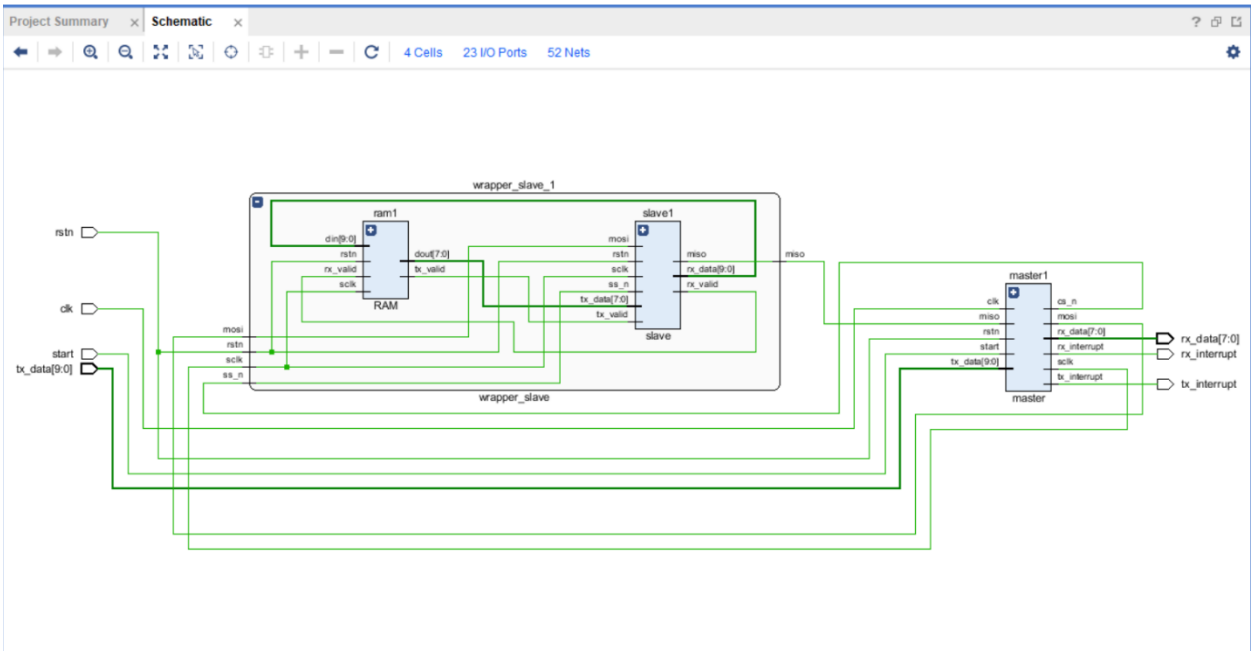
- The Master issues this command to trigger the read operation (the 8-bit payload it sends is ignored). The RAM immediately accesses the memory array at the stored read_adr, outputs that data to dout, then Slave send it to master as MISO, then Master collect them in rx_data



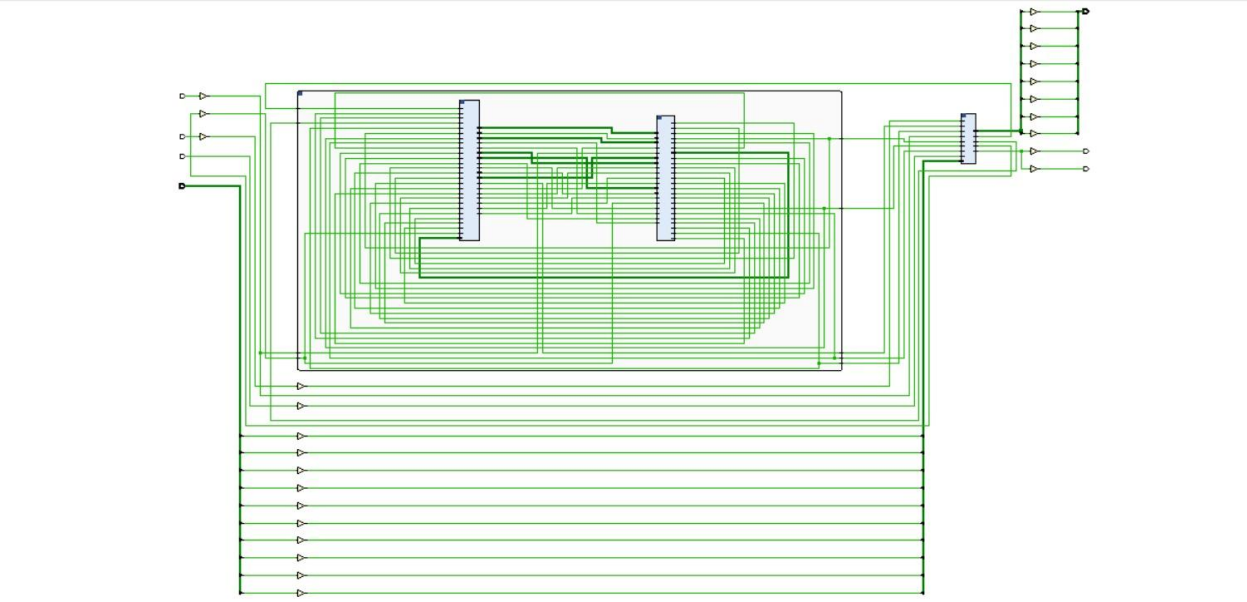


Synthesis

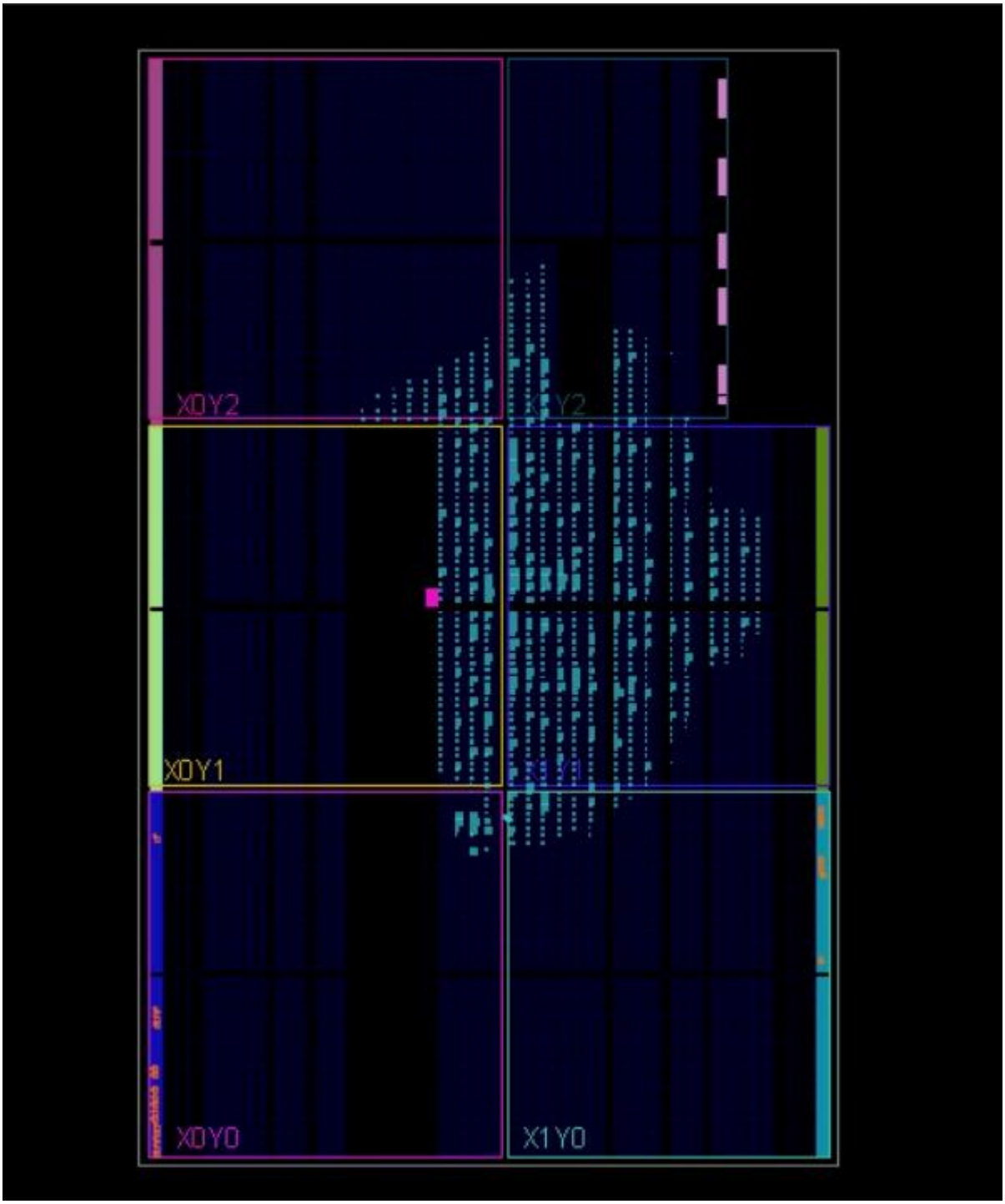
1.RTL Elaboration



2.Synthesis



3.PnR



4.Timing analysis after PnR

- Frequency is used = 200MHz

Design Timing Summary

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	0.629 ns	Worst Hold Slack (WHS):	0.170 ns	Worst Pulse Width Slack (WPWS):	2.000 ns
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	72	Total Number of Endpoints:	72	Total Number of Endpoints:	44

All user specified timing constraints are met.

5.Resource Utilization

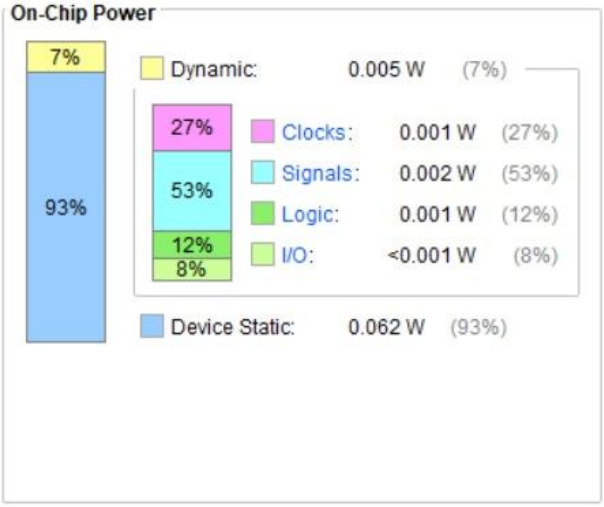
Name	Slice LUTs (20800)	Slice Registers (41600)	F7 Muxes (16300)	F8 Muxes (8150)	Slice (8150)	LUT as Logic (20800)	LUT Flip Flop Pairs (20800)	Bonded IOB (106)	BUFGCTRL (32)
N spi_master_slave	939	2188	272	136	2041	939	85	23	2
master1 (master)	47	43	0	0	18	47	33	0	0
wrapper_slave_1 (wra...	892	2145	272	136	2023	892	52	0	0

6.Power Consumption

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power:	**0.066 W**
Design Power Budget:	**Not Specified**
Power Budget Margin:	**N/A**
Junction Temperature:	**25.3°C**
Thermal Margin:	74.7°C (14.9 W)
Effective ϑJA:	5.0°C/W
Power supplied to off-chip devices:	0 W
Confidence level:	Low

Launch Power Constraint Advisor to find and fix invalid switching activity



Conclusion

- In this project, a fully functional SPI communication system was designed and verified at the RTL level using Verilog HDL. The system features a configurable Master, a Slave, and an integrated RAM interface capable of executing precise serial memory operations. Driven by a modular and parameterized design methodology, the architecture is easily adaptable to diverse FPGA applications. Comprehensive simulation confirmed the reliability of all core functions, including clock domain management, packet decoding, and data transmission. Ultimately, this project serves as a scalable baseline for advanced future hardware developments, including multiple slave architectures, burst transfers, and FIFO integration.